

Abstract

Sign language is the primary mode of communication for the hearing and speech-impaired community. However, a significant communication gap exists between sign language users and those unfamiliar with it.

The lack of widespread sign language knowledge among the general population leads to difficulties in communication for people with hearing disabilities. The **objective** of this project is to design a **real time AI-powered sign language translator system** that accurately recognizes sign language gestures and translates them into textual and auditory output, enabling seamless interaction.

The system uses a deep learning model trained on a sign language dataset. **Computer vision techniques**, including **OpenCV** and **Media Pipe**, are employed for hand tracking and gesture recognition. A **Convolutional Neural Network (CNN)** classifies hand gestures, which are then mapped to their corresponding textual and spoken outputs using **Google Text-to-Speech (gTTS)**. The model is integrated into mobile application.

Initial testing demonstrates high accuracy in recognizing static hand gestures, with real-time processing speeds optimized for smooth interaction. Challenges include **dynamic gesture recognition** and **background noise interference**, which are addressed through dataset expansion and model refinement.

The **AI-powered sign language translator** provides an efficient way for sign language users to communicate with non-signers. Future improvements include **incorporating dynamic sign recognition** and **multilingual support** to enhance accessibility and usability.

TABLE OF CONTENTS

<u>Contents</u>	<u>Page No.</u>
➤ Certificate	i
➤ Acknowledgement	ii
➤ Abstract	iii

<u>Chapter No.</u>	<u>Topics</u>	<u>Page No.</u>
1.	INTRODUCTION	1
2.	OBJECTIVES	2
3.	TECHNOLOGIES USED	3
4.	METHODOLOGY	4
5.	SOURCE CODE	5-7
6.	CHALLENGES AND LIMITATIONS	8
7.	FUTURE ENHANCEMENT	9
8.	CONCLUSION	10
9.	REFERENCES	11

INTRODUCTION

Communication is one of the most fundamental human needs, and for individuals with hearing or speech impairments, sign language serves as a vital mode of expression. However, the majority of the population is not proficient in sign language, which creates a communication gap between the hearing-impaired and the rest of society. To bridge this gap, this project presents an **AI-Based Sign Language Translator** that leverages **computer vision** and **machine learning** to convert hand gestures into readable text in real-time.

The core idea of the project is to recognize sign language **gestures** using a webcam and translate them into **English alphabets or words**. By utilizing frameworks such as **MediaPipe** for real-time hand tracking and **OpenCV** for image processing, the system captures hand landmarks and feeds them into a classification algorithm that identifies the corresponding sign.

OBJECTIVES

The main goal of this project is to create a **real-time sign language translator** that is accurate, fast, and user-friendly. The specific objectives are:

- ❖ To detect hand gestures using a webcam and computer vision techniques.
- ❖ To classify the detected gestures into corresponding alphabetic characters or words.
- ❖ To display the translated text on the screen in real-time.
- ❖ To make the application accessible and usable by individuals with limited technical skills.

TECHNOLOGIES USED

This project is built using a combination of advanced **AI** and **computer vision tools**:

- ❖ **Python:** Chosen for its simplicity and vast library support.
- ❖ **OpenCV:** Used for image and video processing to capture hand movements from the webcam.
- ❖ **MediaPipe:** A framework by **Google** that enables real-time hand tracking by detecting **21 key** landmarks on the hand.
- ❖ **NumPy:** For numerical operations and processing of landmark coordinates.
- ❖ **Machine Learning (Optional): TensorFlow/Keras** can be used to train a model for more complex gesture recognition.
- ❖ **Tkinter/Streamlit:** For creating a graphical user interface (optional for frontend display).

METHODOLOGY

Step 1: Data Collection:

Collected various hand gestures for letters A-Z (or specific signs).

Annotated data for training a classification model (if applicable).

Step 2: Hand Landmark Detection:

Used **MediaPipe**'s hand tracking module to extract 21 landmark points from each frame.

Step 3: Feature Extraction:

Calculated distances and angles between landmarks.
Normalized the input for consistency.

Step 4: Gesture Classification:

For basic gestures, used rule-based logic.

For complex gestures, trained a machine learning model.

Step 5: User Interface:

Developed a simple interface showing live video, detected gesture, and corresponding text.

SOURCE CODE

datacollection.py

```
import cv2
import os
from cvzone.HandTrackingModule import HandDetector
import numpy as np
import math
import time
while True:
    x=input("Enter name of the sign : ")
    folder=f"C:\\Users\\Aakash Adhikary\\Desktop\\sign language\\Data\\{x}"
    # Create folder if it doesn't exist
    if not os.path.exists(folder):
        os.makedirs(folder)
    cap=cv2.VideoCapture(0)
    detector=HandDetector(maxHands=1)
    offset=20
    imgSize=300
    counter=0
    while True:
        success,img=cap.read()
        hands,img=detector.findHands(img)
        if hands:
            hand=hands[0]
            x,y,w,h=hand['bbox']
            imgWhite=np.ones((imgSize,imgSize,3),np.uint8)*255
            imgHeight, imgWidth = img.shape[:2]
            x1 = max(x - offset, 0)
            y1 = max(y - offset, 0)
            x2 = min(x + w + offset, imgWidth)
            y2 = min(y + h + offset, imgHeight)
            imgCrop = img[y1:y2, x1:x2]
            imgCropShape=imgCrop.shape
            aspestratio=h/w
            if aspestratio>1:
                k=imgSize/h
                wCal= math.ceil(k*w)
                imgResize=cv2.resize(imgCrop,(wCal,imgSize))
                imgResizeShape=imgResize.shape
                wGap=math.ceil((imgSize-wCal)/2)
```

```

        imgWhite[ : ,wGap : wCal+wGap]=imgResize
    else :
        k=imgSize/w
        hCal= math.ceil(k*h)
        imgResize=cv2.resize(imgCrop,(imgSize,hCal))
        imgResizeShape=imgResize.shape
        hGap=math.ceil((imgSize-hCal)/2)
        imgWhite[hGap : hCal+hGap , :]=imgResize
    cv2.imshow('ImageCrop',imgCrop)
    cv2.imshow('ImageWhite',imgWhite)
    cv2.imshow('Image',img)
    key=cv2.waitKey(1)
    if key== ord('s'):
        counter+=1
        cv2.imwrite(f'{folder}\\Image_{time.time()}.jpg',imgWhite)
        print(counter)
    elif key == ord('q'): # Press 'q' to quit
        print("Quitting...")
        break
    q=int(input("Press 1 to continue to another sign \n Press 0 to quit : "))
    if q!=1:
        break

```

test.py

```

import cv2
from cvzone.HandTrackingModule import HandDetector
from cvzone.ClassificationModule import Classifier
import numpy as np
import math
cap = cv2.VideoCapture(0)
detector = HandDetector(maxHands=1)
classifier = Classifier(r"C:\Users\Aakash
Adhikary\Downloads\Model2\keras_model.h5", r"C:\Users\Aakash
Adhikary\Downloads\Model2\labels.txt")
offset = 20
imgSize = 300
counter = 0
labels = ["Hello", "I love you", "Thank you", "A"]
while True:
    success, img = cap.read()
    imgOutput = img.copy()
    hands, img = detector.findHands(img)

```



```
if hands:
```

```
    hand = hands [0]
```

```
    x, y, w, h = hand['bbox']
```

```
    imgWhite = np.ones((imgSize, imgSize, 3), np. uint8) *255
```

```
imgHeight, imgWidth = img.shape[:2]
```

```
    x1 = max (x - offset, 0)
```

```
    y1 = max (y - offset, 0)
```

```
    x2 = min (x + w + offset, imgWidth)
```

```
    y2 = min (y + h + offset, imgHeight)
```

```
    imgCrop = img[y1: y2, x1:x2]
```

```
    imgCropShape = imgCrop.shape
```

```
    aspectRatio = h / w
```

```
    if aspectRatio > 1:
```

```
        k = imgSize / h
```

```
        wCal = math.ceil(k * w)
```

```
        imgResize = cv2.resize(imgCrop, (wCal, imgSize))
```

```
        imgResizeShape = imgResize.shape
```

```
        wGap = math.ceil((imgSize-wCal)/2)
```

```
        imgWhite[:, wGap: wCal + wGap] = imgResize
```

```
        prediction, index = classifier.getPrediction(imgWhite, draw= False)
```

```
        print (prediction, index)
```

```
    else:
```

```
        k = imgSize / w
```

```
        hCal = math.ceil(k * h)
```

```
        imgResize = cv2.resize(imgCrop, (imgSize, hCal))
```

```
        imgResizeShape = imgResize.shape
```

```
        hGap = math.ceil((imgSize - hCal) / 2)
```

```
        imgWhite[hGap: hCal + hGap, :] = imgResize
```

```
        prediction, index = classifier.getPrediction(imgWhite, draw= False)
```

```
cv2.rectangle(imgOutput, (x-offset, y-offset-70), (x -offset+400, y -  
    offset+60-50), (0,255,0), cv2.FILLED)
```

```
cv2.putText(imgOutput, labels[index], (x, y-30),
```

```
    cv2.FONT_HERSHEY_COMPLEX,2, (0,0,0),2)
```

```
cv2.rectangle(imgOutput, (x-offset,y-offset), (x + w + offset, y+h + offset),  
    (0,255,0),4)
```

```
cv2.imshow('ImageCrop', imgCrop)
```

```
cv2.imshow('ImageWhite', imgWhite)
```

```
cv2.imshow('Image', imgOutput)
```

```
cv2.waitKey(1)
```

CHALLENGES & LIMITATIONS

❖ **Lighting and Background Sensitivity:**

The accuracy of hand detection relies heavily on consistent and proper lighting. In poorly lit environments or when the background is cluttered, the **MediaPipe** model may struggle to identify hand landmarks accurately.

❖ **Single-Hand Detection Limitation:**

The current implementation often focuses on recognizing gestures made with a single hand. Many sign languages involve two-hand signs, and detecting and interpreting them simultaneously is more complex and currently not supported.

❖ **Static Gesture Limitation:**

This version only supports static gestures (e.g., alphabets or predefined signs). Dynamic gestures, which involve movement over time (e.g., waving for “hello” or signing full words/sentences), are not yet implemented.

❖ **Limited Vocabulary:**

The system currently recognizes a limited number of gestures. It cannot interpret contextual meanings or full sign language grammar yet, which is essential for real-world conversation.

FUTURE ENHANCEMENT

❖ **Support for Dynamic Gestures (Motion-Based Signs):**

Implementing dynamic gesture recognition, where the system can detect movements over time using sequence models like **LSTM (Long Short-Term Memory)**.

❖ **Incorporation of Two-Handed Sign Recognition:**

The future version could implement dual-hand tracking and detection.

❖ **Text-to-Speech Integration (Speech Output):**

To complete the communication loop, the system can be enhanced to convert translated text into speech using a **TTS (Text-to-Speech)** engine like **pyttsx3**, **Google Text-to-Speech**, or **Amazon Polly**.

❖ **Multilingual Sign Language Support:**

Future updates could include support for multiple sign languages, with language selection options, making the system accessible to a global audience.

❖ **Mobile App Development and Deployment:**

Transforming this desktop-based system into a mobile application using frameworks like **Flutter**, **React Native**, or **Android Studio (Kotlin/Java)** would increase accessibility.

❖ **Custom Gesture Training Mode:**

Allow users to train the system with their own custom gestures.

CONCLUSION

This project successfully demonstrates the potential of artificial intelligence in bridging the communication gap between the hearing and speech-impaired community and the rest of society. By developing an **AI-based sign language translator**, we were able to create a system capable of recognizing hand gestures and translating them into corresponding text or speech in real-time. The integration of **computer vision** and **deep learning algorithms** proved effective in ensuring high accuracy and responsiveness.

While the translator performs well under controlled conditions, there are still limitations to be addressed, such as variations in lighting, background noise, and different user hand sizes. These challenges highlight opportunities for further development. Future work could focus on expanding the dataset to include more sign languages and gestures, improving model robustness in diverse environments, and optimizing performance for mobile and low-resource devices.

Overall, this project lays a solid foundation for inclusive communication tools, contributing meaningfully to accessibility and social integration for individuals who rely on sign language.

REFERENCES

❖ **Books:**

- **Pajankar, A. (2020).** *Hands-On Computer Vision with OpenCV & Python*. BPB Publications.
- **Goodfellow, I; Bengio, Y; & Courville, A. (2016).** *Deep Learning*. MIT Press.
- **Solem, J. E. (2012).** *Programming Computer Vision with Python*. O'Reilly Media.

❖ **Research Papers:**

- **Jungpil Shin Akitaka Matsuoka Md. Al Mehedi Hasan and Azmain Yakin Srizon** ,“*American Sign Language Alphabet Recognition by Extracting Feature from Hand Pose Estimation.*” Published: 31 August 2021
- **Zhang, F.; Bazarevsky, V.; Vakunov, A.; Tkachenka, A.; Sung, G.; Chang, C.L.; Grundmann, M.** *Mediapipe hands: On-device real-time hand tracking*. arXiv 2020, arXiv:2006.10214.